DESENVOLVIMENTO BACK-END

Introdução ao Ecossistema e à Arquitetura Server-Side

Unidade I · Do toque em "Entrar" até o banco de dados

Back-end · Cliente-servidor · HTTP/HTTPS · Runtimes · Pacotes · Variáveis de ambiente

🎯 Objetivos de aprendizagem

🧱

Compreender

o que é o back-end e o modelo cliente-servidor

⚖️

Diferenciar

servidor web de servidor de aplicação

🔍

Explicar

o caminho de uma requisição, do DNS ao banco

📨

Identificar

métodos, cabeçalhos, status e o papel do HTTPS

📦

Relacionar

runtime, framework, pacotes e versionamento semântico

🔐

Analisar

o uso de variáveis de ambiente e o cuidado com segredos

📱 O que acontece quando você toca em "Entrar"?

Em **menos de um segundo**:

- seus dados **viajam pela internet** até outro estado ou país
- um programa **verifica quem você é**
- ele **consulta um banco de dados**
- e **devolve uma resposta** ao aplicativo

Esse programa **sem tela nem botões** é o **back-end**.

🍽️

Analogia: o restaurante

O **salão** (mesas, cardápio, garçons) é o **front-end**. A **cozinha** é o **back-end**: ingredientes são os **dados**, receitas são as **regras de negócio** e cozinheiros são o **código**.

O cliente **não entra na cozinha**, mas a qualidade do prato depende **inteiramente dela**.

🧱 Front-end × Back-end

🖥️

Front-end · o salão

O que o usuário **vê e toca**: telas, botões, formulários e animações, executados no **navegador** ou no **app**.

⚙️

Back-end · a cozinha

Roda no **servidor**: recebe pedidos, aplica **regras de negócio**, acessa **dados** e devolve respostas.

💡 **Conceito-chave:** back-end é o conjunto de programas executados no lado do servidor (**server-side**) que recebem requisições, aplicam regras de negócio, persistem e recuperam dados, integram-se a outros sistemas e **garantem a segurança** da aplicação.

É o back-end que garante que a nota lançada pelo professor apareça **corretamente no boletim** do aluno.

🔁 O modelo cliente-servidor

- O **cliente inicia** a comunicação, pedindo algo
- O **servidor aguarda** pedidos e os atende
- Um mesmo back-end atende **vários clientes**
- O HTTP é **sem estado** (*stateless*): cada requisição é **independente**

🌐
Navegador
portal do aluno

📱
App
celular do professor

💳
Outro sistema
plataforma de pagamentos

⚙️
Back-end
a mesma API para todos

💡 **Sem estado** facilita a escala (qualquer servidor atende qualquer pedido), mas exige **cookies, sessões ou tokens** para reconhecer o usuário entre uma requisição e outra.

⚖️ Servidor web × servidor de aplicação

	🌐 Servidor web	⚙️ Servidor de aplicação
Exemplos	Nginx, Apache	PHP-FPM, processo Node.js
Especialidade	tarefas de **rede**	executar a **lógica** do programa
Faz	aceita conexões, negocia **TLS**, comprime, entrega **arquivos estáticos**	roda o código, aplica regras, **consulta o banco**
Escala	de forma **independente**	de forma **independente**

💡 "Servidor" tem dois sentidos: **físico** (a máquina, real ou virtual) e **lógico** (o programa que escuta uma porta, como a 443). Ao dizer "servidor web", quase sempre falamos do **lógico**.

🔎 O fluxo de uma requisição



- O cliente **descobre o IP** do servidor consultando o **DNS**
- Abre uma **conexão segura** e envia a **requisição HTTP**
- A aplicação **executa o código**, consulta o **banco** e a resposta **faz o caminho de volta**

Uma lentidão pode estar em **qualquer etapa**: rede, processamento ou, principalmente, **banco de dados**.

💻 O menor back-end possível em PHP

```php
<?php
// public/index.php — rode com: php -S localhost:8000 (só para estudo!)
header('Content-Type: application/json; charset=utf-8');   // a resposta será JSON
$metodo  = $_SERVER['REQUEST_METHOD'];                      // 1. lê o verbo: GET, POST...
$caminho = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH); // 2. lê o caminho pedido
$cursos  = [['id' => 1, 'nome' => 'Engenharia de Software'],  // dados fixos, por enquanto
            ['id' => 2, 'nome' => 'Ciência de Dados']];

if ($metodo === 'GET' && $caminho === '/api/cursos') {      // 3. roteamento: decide o que fazer
    echo json_encode($cursos, JSON_UNESCAPED_UNICODE);      // 4. representação em JSON (200 OK)
    exit;                                                    //    encerra a resposta aqui
}
http_response_code(404);                                     // 5. código de status: não encontrado
echo json_encode(['erro' => 'Rota não encontrada']);         //    corpo explicando o erro
```

📌 **O que demonstra:** todo back-end **lê método e caminho**, faz o **roteamento**, produz uma **representação** dos dados e define o **código de status**. Frameworks fazem o mesmo, de forma organizada e segura. ⚠️ O `php -S` atende uma requisição por vez: **nunca em produção**.

📨 A anatomia de uma mensagem HTTP

```
GET /api/cursos HTTP/1.1          # linha inicial: método, caminho, versão
Host: localhost:8000              # cabeçalho: qual servidor
Accept: application/json          # cabeçalho: formato aceito
                                  # linha em branco: fim dos cabeçalhos
HTTP/1.1 200 OK                   # linha de status: código + frase
Content-Type: application/json    # cabeçalho: formato do corpo
                                  # linha em branco
[{"id":1,"nome":"Engenharia..."}] # corpo: os dados em JSON
```

- **Requisição:** linha inicial, **cabeçalhos**, linha em branco e **corpo** opcional
- **Resposta:** começa com a **linha de status**
- **URL:** esquema, **host**, porta, **caminho**, *query string* e fragmento

💡 Rode `curl -v` na sua API: linhas com `>` foram **enviadas** pelo cliente; com `<`, **recebidas** do servidor. O **fragmento** (`#...`) nunca chega ao servidor.

🗣️ Métodos e cabeçalhos

O método expressa a intenção

- `GET` consulta · `POST` cria
- `PUT` substitui · `PATCH` altera parte
- `DELETE` remove
- **Seguro:** não altera o servidor (`GET`)
- **Idempotente:** repetir dá o **mesmo efeito** (`PUT`, `DELETE`)

Cabeçalho	Função
`Content-Type`	formato do corpo
`Accept`	formatos aceitos
`Authorization`	credenciais (`Bearer`)
`Cookie` / `Set-Cookie`	envio e criação de cookies
`Cache-Control`	regras de cache

💡 O **corpo** carrega os dados: vazio no `GET`, com a representação do recurso em `POST`, `PUT` e `PATCH`, quase sempre em **JSON**.

🚦 Códigos de status: o resultado em 3 dígitos

Classe	Significado	Exemplo
1xx	informativo	—
2xx	sucesso	`200 OK`
3xx	redirecionamento	HTTP ➜ HTTPS
4xx	erro do **cliente**	`404` rota inexistente
5xx	erro do **servidor**	`500` falha interna

🍪

E o "estado"?

O servidor envia `Set-Cookie`; o navegador devolve `Cookie` nas próximas requisições. Atributos **HttpOnly**, **Secure** e **SameSite** deixam o cookie mais seguro.

O código de status faz parte do **contrato da API**: é por ele que o cliente decide **o que mostrar ao usuário**.

🔒 HTTPS, TLS e a evolução do HTTP

🙈

Confidencialidade

ninguém no caminho **consegue ler** o conteúdo

🧩

Integridade

ninguém **altera** a mensagem sem ser percebido

🪪

Autenticidade

o **certificado digital** prova que o servidor é o verdadeiro

HTTP/1.1 · 1997
conexões persistentes

➜

HTTP/2 · 2015
binário, multiplexação

➜

HTTP/3 · 2022
QUIC sobre UDP

💡 **HTTPS = HTTP sobre TLS.** Em produção: redirecione todo HTTP para HTTPS e use **HSTS**. A **semântica** (métodos, cabeçalhos, status) nunca mudou: só o **transporte** evoluiu.

🍽️ Os conceitos da aula no dia a dia

Conceito	No dia a dia	Na API acadêmica
Cliente-servidor	cliente faz o pedido, a cozinha prepara	o app pede `GET /api/cursos`
DNS	agenda de contatos: nome ➜ número	`api.faculdade.edu.br` ➜ IP
Status HTTP	o garçom avisa: "saiu", "acabou", "a cozinha travou"	`200`, `404`, `500`
HTTPS/TLS	envelope **lacrado** e com remetente comprovado	tokens e notas trafegam cifrados
Lock file	receita com a **marca exata** de cada ingrediente	`composer.lock` igual em todos os ambientes

💡 Use as analogias para **lembrar** da ideia, mas explique sempre com o **termo técnico**: é ele que aparece na documentação e nas entrevistas.

🧰 Linguagem, runtime, biblioteca e framework

✍️

Linguagem

a **sintaxe** e as regras de escrita: PHP, JavaScript

⚙️

Runtime

quem **executa** o código: Zend Engine, Node.js, JVM

📚

Biblioteca

código reutilizável que **você chama**: gerar JWT

🏗️

Framework

define a estrutura e **chama o seu código**: Laravel, Express

Modelo de execução	PHP com PHP-FPM	Node.js
Ciclo de vida	um processo por requisição, **memória liberada** ao final	processo de **longa duração**
Concorrência	**pool** de processos em paralelo	**event loop** assíncrono

"Você chama a biblioteca, mas **o framework chama você**": isso é **inversão de controle**.

📦 Gerenciadores de pacotes e versionamento semântico

Ecossistema	Gerenciador	Manifesto / trava
PHP	Composer	`composer.json` / `.lock`
Node.js	npm, pnpm	`package.json` / `-lock.json`
Java	Maven, Gradle	`pom.xml` / `build.gradle`

🏷️

SemVer: MAJOR.MINOR.PATCH

MAJOR quebra compatibilidade · **MINOR** novidade compatível · **PATCH** correção. `"firebase/php-jwt": "^6.10"` aceita qualquer **6.x ≥ 6.10**, mas **nunca a 7.0**.

💡 **Boa prática:** versione o `composer.lock` e use `composer install` no servidor. A pasta `vendor/` (ou `node_modules/`) **nunca** vai para o Git.

🔐 Variáveis de ambiente e segredos

❌

Sem separar configuração

- senha do banco **escrita no código**
- `.env` enviado ao **GitHub público**
- `APP_DEBUG=true` em produção
- mesma chave em **todos os ambientes**

✅

Com variáveis de ambiente

- valores no `.env`, lidos **só em** `config/`
- no código, apenas `config('jwt.ttl_minutos')`
- `.env.example` versionado, com valores **fictícios**
- produção usa um **cofre de segredos**

Configuração é o que varia entre ambientes; **código** é o que não varia. Vazou um segredo? **Revogue e troque** na hora.

🌍 Aplicação em sistemas reais

🎓

Portal acadêmico

Nginx + **PHP-FPM** + PostgreSQL; um `.env` diferente por ambiente

🏦

App de banco

HTTPS obrigatório, HSTS e segredos em **cofre auditado**

💬

Chat e notificações

Node.js e seu event loop para muitas operações de E/S

Nova versão → **Sem pacotes de dev** → **Cache de configuração** → **Debug desligado** → **HTTPS + HSTS ✔**

💡 A **configuração faz parte da arquitetura e da segurança**: esse roteiro costuma rodar automaticamente em um pipeline de **CI/CD**.

Síntese dos conceitos

1. **Back-end** = a cozinha: regras, dados e segurança
2. **Cliente-servidor**: o cliente pede, o servidor atende
3. **Servidor web** (rede) × **servidor de aplicação** (lógica)
4. **Fluxo**: Cliente ➜ DNS ➜ Web ➜ Aplicação ➜ Banco

1. **HTTP**: método, cabeçalhos, corpo e **status**
2. **HTTPS/TLS**: confidencialidade, integridade, autenticidade
3. **Runtime + pacotes + SemVer** organizam o ecossistema
4. **Configuração fora do código**; segredos em **cofre**

O back-end é o **contrato invisível** do sistema: quem domina o **caminho da requisição** domina qualquer framework.

📝 Reflexão

🧩 Desafio: complete a primeira rota

```php
// routes/api.php — GET /api/saude
Route::______('/saude', function () {        // qual verbo HTTP?
    return response()->json([
        'status'    => 'ok',
        'aplicacao' => ______('app.name'),   // ler do .env ou da config?
    ]);                                       // qual status sai aqui?
});
```

💬 Reflita

1. Se o **banco cair**, o usuário deve ver uma tela em branco, o **erro do SQL** ou uma mensagem amigável?
2. Por que o `composer.lock` vai para o Git e a pasta `vendor/` **não**?
3. Na API acadêmica, quais dados **nunca** podem trafegar sem **HTTPS**?